Calibrated Hallucination Risk Estimation in Large Language Models Using Evidence-Consistent XGBoost

Md. Atikur Rahaman [a,*], Saiful Alam Sabbir [a], MD. Miraz Ahamed [a], Ohidujjaman Tuhin [a]

[a] *Computer Science and Engineering, United International University, Dhaka 1212, Bangladesh*

ARTICLE INFO

Keywords:
Distribution shift
Evidence-consistent XGBoost
Explainable AI
Hallucination risk estimation
Probability calibration
Question answering
SHAP

Abstract

Large language models answer questions in fluent text, and some answers are wrong in ways the evidence does not support. Detectors for false information have many limitations. For example, they require a large judge model; a computer with graphics capability to run an encoder classifier; feature-based detectors provide little information regarding how well their estimates are calibrated or how well these detectors perform outside the domain of their training data. Therefore, this study was designed to see if a simple black-box model could accurately predict the likelihood of hallucinations from the candidate answer itself along with the question asked and the supporting evidence provided. The proposed method, Evidence Consistent eXtreme Gradient Boosting (EC-XGB), uses an ensemble of decision trees trained over 35 features derived from the input evidence. Eight of the 35 features represent the extent to which the claims are supported. Because the benchmark is nearly fully saturated, the useful gain comes from better reliability during out-of-domain shifts. A tree model using only explicit evidence features requires no model access and remains inexpensive to interpret. Results show that EC-XGB achieves an F1 of 0.9855 and an AUROC of 0.9984 on the HaluEval question answering task, competitive with a conventional XGBoost model. When evaluating results on task type categories (RAGTruth) included in the training set, the AUROC increases from 0.475 to 0.582, while the proportion of natural RAGTruth answers identified as at-risk decreases from 99.9% to 61.3%. Performance on a QA evaluation task not seen during training is consistent with random guessing (i.e., an F1 near 0.30). Finally, a model-specific display calibrator fit to labeled target rows reduces the average target calibration error from 0.726 to 0.126.

1. Introduction

A number of researchers have examined how to apply machine learning techniques to assess the likelihood of a response being a hallucination. One such technique is SelfCheckGPT, which uses several generated answers and assesses their similarity to determine if there has been a hallucination [1]. Another technique is Luna, where a smaller evaluation model is used instead of a larger judge model [2]. A third technique is semantic entropy, which identifies meaning-based groupings of similar responses [3]. A hybrid framework combines frozen encoder features with lightweight classifiers and reports strong benchmark scores [4]. Judges carry high computational costs and encoder models are hard to interpret. Two recent reviews have grouped approaches to detecting hallucinations into three categories including feature-based detectors, encoder-based detectors, and combinations of the above [5, 6]. Both reviews identify evaluation and reliability as two major areas that require additional research, and a third review determines that reliability is an area of concern as well [7].

Six recent studies address the same topic of assessing hallucinations. HaluAgent demonstrated that small language models can select appropriate tools and detect multiple forms of hallucinations, but it required multiple tool calls for each answer [8]. Valentin et al. proposed a calibrated multi-score detector with a detailed cost analysis, but the detector relies heavily on large language model (LLM) scoring for its primary source of information [9]. Sundaragiri et al. proposed a system that uses retrieval, natural language inference (NLI) verification, and an XGBoost classifier, and they report only classification metrics [10]. Haq et al. assigned SHAP-LIME attributions to each word, but did not test whether those values remained consistent across words [11]. Deng et al. found that detectors degrade once they leave the domain in which they were trained, and their method focuses on multi-turn dialogue signals [12]. FaithBench shows that modern LLMs still produce hard summarization hallucinations, and it does not assess calibration risks [13].

*Corresponding author: Md. Atikur Rahaman
Email addresses: mrahaman2310298@bscse.uiu.ac.bd (Md.A. Rahaman); sabbir2310105@bscse.uiu.ac.bd (S.A. Sabbir); mahamed2310524@bscse.uiu.ac.bd (MD.M. Ahamed)

The existing literature leaves four issues unaddressed. Calibration is infrequently reported for lightweight detectors, explanation quality is rarely measured, cross-domain behavior is rarely tested, and error patterns are rarely analyzed case by case. This paper addresses all four issues by developing a black-box model that takes a query q, a reference context c, and a candidate answer a, and outputs $P(H \mid q, c, a)$. The model, evidence-consistent XGBoost (EC-XGB), consists of 26 evidence-consistency features in seven groups, eight claim-level NLI aggregates, and a source indicator, classified by a tuned XGBoost model. Platt and isotonic calibration yield the probability, SHAP explanations reveal feature contributions, artifact-control baselines and group-aware splits prevent shortcut solutions, and validation-only calibration solves the reliability problem. Transfer limits are established on RAGTruth and FaithBench.

HaluEval QA provides 10,000 paired correct and hallucinated answers for training and in-domain testing [14]. RAGTruth provides natural responses for transfer and recalibration [15]. The scope is question answering with a reference context, and FaithBench serves only as an out-of-domain summarization stress test. EC-XGB reaches an F1 of 0.9855 on HaluEval QA. On natural RAGTruth answers, the flag rate falls from 99.9% to 61.3%, and the AUROC rises from 0.475 to 0.582. Target calibration on 5,034 labeled rows lowers the RAGTruth QA ECE from 0.726 to 0.126. On HaluEval, recall stays above 99% at a five percent false-positive rate, and one analysis takes 61.8 ms at about $0.001 per 1,000 predictions.

2. Conventional Method

2.1. Preliminaries

Fluent hallucinations are those that are not supported by the available evidence. In most cases, detection quality is assessed using labeled benchmarks and a metric, typically precision, recall, F1 or the area under the receiver operating characteristic curve (AUROC). Current test beds include HaluEval [14] and TruthfulQA [16] for hallucination detection. FActScore [17] assesses factual accuracy for long-form generation. For deployment, there are two key evaluation considerations. First, calibration. Only if a risk score has a numeric value that is consistent with the probability of generating a hallucination can that risk score be used effectively. Expected calibration error (ECE) and reliability diagrams are used to show how far off a model is in terms of calibration [18]. Second, the need for interpretability. Users need to understand what specific signal caused their high-risk flag to appear, and this can be done through feature attribution techniques such as SHapley Additive exPlanations (SHAP) for tabular models [19]. This work assesses both of these properties within a black-box setting.

2.2. Baseline Method

The conventional method in this study is a tuned XGBoost classifier [20] over the 26 hand-crafted evidence-consistency features. XGBoost is selected as the baseline because it handles nonlinear interactions, it trains quickly on a tabular design matrix, and its tree structure supports exact SHAP values. XGBoost is a gradient-boosting ensemble, so the model adds decision trees one at a time and each new tree fits the error left by the trees already in the ensemble [20]. Each tree splits the feature space into leaves, and each leaf stores a weight that adds to the running score. The final score passes through a logistic link, so the output is a probability of the hallucinated class.

Let x_i be the feature vector of row i and $y_i \in \{0, 1\}$ its label. The model sums K trees, $z_i = \sum_{k=1}^{K} f_k(x_i)$. The training objective combines a differentiable loss with a regularizer over the trees,

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \left(\gamma T_k + \tfrac{1}{2}\lambda \|w_k\|^2\right), \tag{1}$$

where $\phi = \{f_k\}_{k=1}^{K}$ collects the trees, f_k is the k-th tree with T_k leaves of weight w_k, γ is a leaf-count penalty, and λ is an L2 weight penalty. The loss l is the standard logistic loss and the probability comes from the sigmoid of the summed leaf score, $\hat{y}_i = \sigma(z_i) = 1/(1 + e^{-z_i})$.

Tree depth, learning rate, tree count, subsample rate, and column subsample rate are tuned on the training split, and the selected values and the search protocol are reported in Section 4. This baseline is the reference for every later modification, and Algorithm 1 summarizes its training and inference procedure.

Algorithm 1 Baseline feature-based XGBoost (training and inference)

Require: labeled rows (q_i, c_i, a_i, y_i) and seeds $S = \{42, 123, 456\}$
Ensure: trained model, calibrator, and decision threshold
1: extract the 26 evidence-consistency features for every row
2: tune XGBoost with a randomized search and grouped 5-fold cross-validation
3: **for** each seed $s \in S$ **do**
4: train the classifier on the training split
5: fit Platt calibration and select the decision threshold on the validation split
6: evaluate on the fixed test split
7: **end for**
8: **return** the mean metrics over the seeds

2.3. Limitations and Research Gap

This baseline performs well as a standalone for its specific benchmark, but it is limited. On HaluEval the benchmark has saturated, so the addition of features has little impact on performance. The baseline relies heavily on both answer length and lexical similarity. As demonstrated in Section 5, answer length is essentially an ideal feature to utilize when working with this dataset. However, under domain shift, the raw probabilities lose their reliability. Specifically, the baseline flags virtually all natural answers, and its confidence no longer correlates with the observed error rate.

The benchmark itself represents one limiting factor, since close to 85% of the full HaluEval benchmark was produced by ChatGPT through sampling and filtering [14, 21]. The line separating factual errors from hallucinations is therefore blurred in many instances. Another issue is that the baseline does not include a claim-level evidence signal, was trained on a single corpus, and provides a probability without a method to calibrate it to the target domain or a reliability test for its explanations.

3. Proposed Method

3.1. Working Principle and Task

The entire proposed pipeline is illustrated in Figure 1. The system accepts a question, an optional reference context (evidence), and a candidate answer produced by a black-box LLM, and it produces the calibrated risk probability and per-feature explanations for that answer.

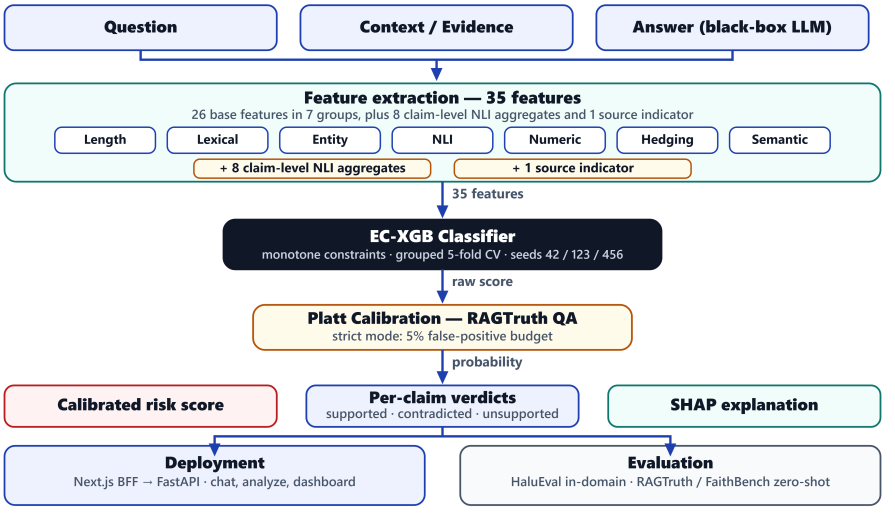


Figure 1: End-to-end architecture of the proposed model. The inputs feed 35 features into the EC-XGB classifier, Platt calibration produces the deployed score, and the outputs are the risk score, the per-claim verdicts, and the SHAP explanations. The system is deployed as a Next.js and FastAPI application and evaluated in-domain on HaluEval and under shift on RAGTruth and FaithBench.

The goal is to estimate the conditional probability $P(\text{hallucinated} \mid q, c, a)$ of hallucination, conditioned on the question q, the reference context c (if applicable), and the candidate answer a produced by a black-box LLM. The

approach does not estimate truth. It flags cases where an answer does not follow from, or contradicts, the provided evidence.

3.2. Features

The model uses 26 base features in seven categories (Table 1) and eight claim-level aggregates (Table 2). For NLI, six signal probabilities are determined using a DeBERTa-v3 cross-encoder [22] trained on SNLI [23] and MultiNLI [24]. These probabilities capture whether a given statement follows logically from another. Semantic similarity uses MiniLM sentence embeddings [25, 26], entities are detected by spaCy named entity recognition [27], and numeric and hedging features are found through regular expression matching or a small hedging lexicon.

Table 1
Base feature groups (26 features total).

Group	Features	Source
Length/style	n_chars, n_words, n_sentences, avg_word_len	regex
Lexical	overlap vs. context & question, Jaccard ×2	token sets
Entity	entity counts, overlap ratio, novel-entity ratio	spaCy NER
NLI	entail/contradict/neutral × 2 directions	cross-encoder NLI
Numeric	number counts, overlap ratio, novel numbers	regex
Hedging	hedge count, hedge density	lexicon
Semantic	cosine(context,answer), cosine(question,answer)	MiniLM

Table 2
Claim-level NLI aggregates (8 features). The answer is split into at most six atomic clauses, with conjunctions such as "and" or "but" broken into separate clauses, each clause is scored against its four best-matching context sentences, and the pair probabilities are aggregated.

Signal	Features	Source
Contradiction	max and mean contradiction, contradicted-claim ratio	NLI cross-encoder
Support	supported-claim ratio, unsupported-claim ratio, min and mean entailment	NLI cross-encoder
Decomposition	number of claims	sentence splitter

3.3. Evidence-Consistent XGBoost

EC-XGB alters three key aspects of the conventional XGBoost learner. These changes allow each component of the learning process to be assessed individually. First, raw-length counts are replaced with log-scaled values, and monotonicity constraints ensure that risk cannot decrease as contradiction or novelty rises and cannot increase as entailment, overlap, or semantic similarity rises. Raw-count length is replaced with its log-scaled value, $\ell' = \log(1+\ell)$.

Second, the eight claim-level aggregates in Table 2 are added to the evidence signals that the deployed verification layer reports, so the classifier sees the same signals. Consider an answer composed of K claims, with $C(k)$ the contradiction and $E(k)$ the entailment of claim k, and let v_k be the verdict of claim k. The aggregate vector is

$$\psi(x) = [\max_k C(k), \overline{C}, r_{\text{con}}, r_{\text{sup}}, r_{\text{unsup}}, \min_k E(k), \overline{E}, K], \tag{2}$$

$$r_{\text{con}} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{1}[v_k = \text{contradicted}], \quad r_{\text{sup}} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{1}[v_k = \text{supported}], \tag{3}$$

$$r_{\text{unsup}} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{1}[v_k = \text{unsupported}]. \tag{4}$$

Here $\overline{C}$ and $\overline{E}$ are the means of $C(k)$ and $E(k)$ over the claims. Let $\phi'(x)$ be the complete model input and $\tilde{\phi}(x)$ the remaining 24 base features excluding the two log-scaled length columns; there are 35 input features in total (24 base + 2 log-scaled length + 8 claim + 1 source).

Third, a source indicator $s \in \{0, 1\}$ is added as a training column for RAGTruth summarization and data-to-text. Because all RAGTruth QA rows are excluded during training, the QA setting remains zero-shot. The deployed display score is fitted on labeled target rows and is therefore not zero-shot. The deployed checkpoint incorporating all three modifications is referred to as EC-XGB.

Algorithm 2 Evidence-consistent XGBoost (EC-XGB) training and deployment

Require: labeled HaluEval QA rows, RAGTruth non-QA rows, seeds S
Ensure: deployed model, display calibrator, source threshold, target threshold
1: extract the 26 base features and the eight claim-level aggregates
2: log-scale the length features and set the monotone constraints
3: **for** each seed $s \in S$ **do**
4: build the 35-feature input with the source indicator
5: train XGBoost on HaluEval plus RAGTruth non-QA rows, and keep every RAGTruth QA row out of training
6: fit the display calibrator on the RAGTruth QA calibration split (P1)
7: select the source-only threshold on the HaluEval validation split (P0)
8: select the target threshold on the same calibration split at five percent FPR (P2)
9: **end for**
10: **return** the deployed model, calibrator, and thresholds

The contribution is the combination. Each piece is standard on its own, and together they hold the in-domain score, cut the false-alarm rate under shift, and keep the output explainable at the feature level. The contribution is therefore a method and an evaluation protocol rather than a new learning algorithm. The deployed system adds claim-level verification with evidence retrieval and a display calibrator fitted on the target domain. Relative to the closest feature-based detector [10], the difference is the claim-level evidence features and the calibration protocol. Relative to the LLM-scored detector [9] and the tool-based detector [8], the difference is that inference needs no model calls. Their scores come from other corpora and other protocols, so the comparison is about the protocol and not about the numbers. None of the three reports the calibration error or the shifted flag rate that this study measures.

3.4. Calibration and Explainability

Calibrators are fit on the validation split only. Platt scaling fits a sigmoid $p(x) = \sigma(Af(x) + B)$ to the raw score $f(x)$, where A and B are fit by maximum likelihood [28]. Isotonic regression fits a non-decreasing step function g so that $p(x) = g(f(x))$ [29]. Both are compared against the raw model on the test split. The comparison uses expected calibration error over $M = 10$ bins,

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|, \tag{5}$$

where $\text{acc}(B_m)$ and $\text{conf}(B_m)$ are the accuracy and mean confidence in bin B_m, together with the Brier score $\text{BS} = \frac{1}{n}\sum_i (\hat{p}_i - y_i)^2$ and reliability diagrams [18]. A second step targets a new domain. The calibrator can be re-fit on a small labeled sample of the target corpus, and the effect is measured on a disjoint held-out part of the target corpus. The deployed EC-XGB display score follows the same rule: its calibrator is fitted on the 5,034-row RAGTruth QA calibration split, and the source indicator is set to one for natural responses. This target calibration needs labeled target rows and is not zero-shot. The source-only protocol uses the HaluEval validation split for both the calibrator and the threshold.

The global SHAP [19] explanations from a TreeExplainer applied to the raw XGBoost model were used to create the Mean Absolute Importance and Beeswarm Summaries. The local explanations of features that contribute to a particular prediction are provided. Many feature attribution methods can be unreliable so explanation reliability is measured directly [30]. The test answers were changed in six different ways (replacing entity names, numbers, and dates; removing supporting context; adding irrelevant context; and shuffling the clauses). This study quantifies the

reliability by calculating four different measures (the mean absolute score change; the fraction of large score changes; the top-1 flip rate; and the Spearman rank correlation between the original and perturbed scores.)

3.5. System and Reproducibility

These methods are used in a two-tiered web application. The system is a Next.js frontend and a FastAPI backend that loads the model, calibrator, explainer, and feature models at startup. Unless a baseline or ablation variant is named, the results in Section 5 use the EC-XGB classifier and its calibrator.

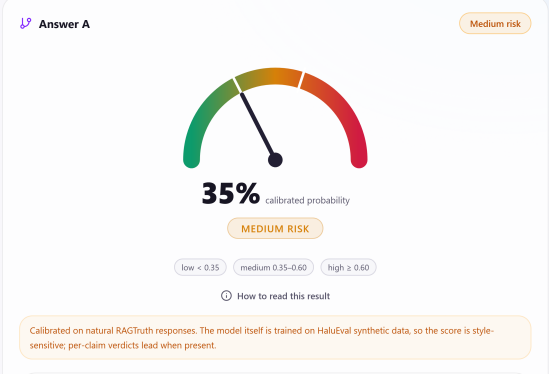
The claim verification layer displays a verdict for each claim and is demonstrated through the interface rather than quantified.

Claims are broken down from the answer text and then checked with sentence-level NLI against one of three evidence sources: the provided context, an indexed document collection, or live web search. Verdicts are made first as support-based, so any supporting sentence provides evidence for a claim, while a contradiction counts only when no supporting sentence exists. A relevance gate is applied next to drop retrieved sentences that do not contain vocabulary related to the claim. The supported, contradicted, and unsupported verdicts are then displayed per claim. For each contradicting claim, the driving evidence sentence is also quoted.

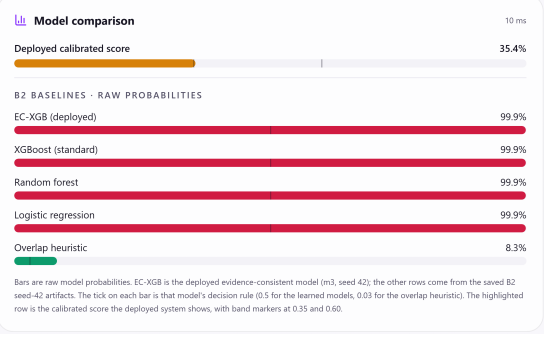
The verdict card sets the risk to high for any contradicted claim, to medium for an unsupported claim, and to low for a fully supported claim. The risk order is consistent with the style sensitivity found in Section 5, where an answer containing complete sentences may cause the calibrated score to reach saturation levels while every individual claim is still supported.

The risk percentage shown on the card is determined by the EC-XGB display calibrator but is modified by the verdict assigned to each individual claim. The 35.4% in Figure 2 is the calibrated score before this adjustment. The claim card marks the contradicted number separately.

Figure 2 presents the analyze page for the hallucinated-number case. The context states 5 days per decade and the answer states 10. The raw EC-XGB probability is 0.999. Without modification by a calibration method, the raw score itself does not represent a usable risk level. The deployed display calibrator indicates 35.4% risk and classifies the answer as moderate risk.



(a) Risk card: display score 35.4% (medium risk).



(b) Model comparison: raw probabilities of the deployed model and the baselines.

Figure 2: Analyze page for the hallucinated-number case, where the context states 5 days per decade and the answer states 10. The risk card shows the calibrated display score, and the comparison panel shows the raw probability of each model. The raw EC-XGB probability is 0.999, while the deployed display calibrator reports 35.4%, which makes the risk level actionable.

`requirements.txt` pins every dependency with exact versions, split indices are saved to `artifacts/split_indices.json` and `.npy`, and `verify_artifacts.py` checks every reported number against the frozen artifacts.

4. Experiments

4.1. Experimental Setup and Conditions

All experiments were run on a Windows-based computer (laptop), with an NVIDIA RTX 3060 6 GB GPU, CUDA 12.8, torch 2.11.0+cu128, and 32 GB of RAM. The programming language used to run the code is Python. The Python version used in this experiment is 3.12. The code utilizes the following packages: scikit-learn 1.9.0, XGBoost 3.3.0, SHAP 0.52.0, spaCy 3.8.14, and sentence-transformers 5.6.1. Each run can be reproduced because of saved split indices and centralized seeds.

A randomized search was performed over 30 hyperparameter configurations with grouped five-fold cross-validation. For the seed-42 conventional model, the configuration has a maximum tree depth of 4, a learning rate of 0.01, 500 trees, a subsampling ratio of 0.9, and a column sampling ratio of 0.7. Each seed and each EC-XGB variant was tuned separately. Seeds 42, 123, and 456 are repeated throughout.

4.2. Dataset

The QA subset of HaluEval [14] serves as the training data. This dataset has 10,000 questions. Each question has a knowledge passage that it is based upon, a correct answer, and a hallucinated version of that answer. Therefore, there are 20,000 rows of labeled data. The 20,000 rows are split 70/15/15 at the group level into 14,000 training, 3,000 validation, and 3,000 test rows. Since both versions of the answer to each question come from the same knowledge passage, the model can easily learn the relationship between the context and the answers, so the split is at the group level.

Transfer was evaluated using two datasets. First, RAGTruth [15]: a collection of responses produced through natural processes that have been annotated with word-level spans by humans. Second, FaithBench [13]: a hallucination-based summarization benchmark based on outputs from modern large language models. The RAGTruth QA task contributed 5,934 rows in 989 groups. Of these, 900 rows in 150 groups form the test set and 5,034 rows form the recalibration set. The groups do not overlap. The summary of the corpora is provided in Table 3.

Table 3
Datasets used for training and evaluation.

Corpus	Rows	Groups	Use
HaluEval QA	20,000	10,000	train / validation / test
RAGTruth QA	5,934	989	held-out + recalibration
RAGTruth full	17,790	2,965	transfer (seen tasks)
FaithBench	750	750	zero-shot

Text is normalized for whitespace. Lexical features work on lowercased token collections. NER and NLI use the original case. Missing context is treated as a special condition. Lexical overlap is 0, and NLI has all probabilities equal to one third. All three of these datasets were developed using their original licensing agreements: HaluEval and RAGTruth (MIT) and FaithBench (CC BY-NC-SA 4.0). The raw data is not redistributed; however, the repository ships the download scripts, the source revisions, and the SHA-256 hashes, which allows regeneration of each of the splits based upon the original repositories.

4.3. Experimental Procedure

Nine methods are tested. The overlap heuristic sets risk to one minus the lexical overlap between the answer and the context. Two additional classifiers were used: a logistic regression on all standardized input features (Table 1) and a random forest with 300 decision trees. The conventional method is XGBoost [20], which uses the 26 base features and is the reference for the proposed model in Section 3. To assess the impact of the feature design alone, three control experiments were run. An NLI-only logistic regression was fit solely on the six NLI probability outputs; an answer-only TF-IDF linear model was fit on the vectorized answers; and a TF-IDF linear model was fit on the entire input. Internal-state detectors need model access and fall outside the black-box setting [31]. The test set contains 1,500 rows per class. A majority classifier that always predicts the negative class therefore scores zero.

A RoBERTa-base classifier fine-tuned on the labeled source data provides the source-only neural evaluation [32]. The classifier receives the answer, question, and context as a single input sequence of up to 256 tokens. The model is trained with binary cross-entropy loss for three epochs. Platt scaling is fitted on the validation split.

Evaluation uses precision, recall, F1, the area under the receiver operating characteristic curve (AUROC), the area under the precision-recall curve (PR-AUC), and the Matthews correlation coefficient (MCC). Calibration is measured with expected calibration error over 10 bins, the Brier score, and negative log-likelihood. Metrics are reported to four decimal places, and latency in milliseconds. Significance is assessed with McNemar's test on the paired seed-42 predictions, bootstrap 95% confidence intervals from 1,000 resamples over question or source groups, and a Wilcoxon signed-rank test across seeds. Rows that represent the same question never occur in both the training and evaluation splits.

4.4. Additional Protocols

Calibration methods were evaluated through a comparison of the three probability sets (raw, Platt, and isotonic) on the test split. The same comparison was performed during the target-domain evaluation, where distribution shift affects calibration [33]. Table 4 defines the three protocols (P0, P1, and P2) and the row split for each protocol.

Table 4
The three evaluation protocols. Only P0 is zero-shot on the target corpus, because P1 and P2 both read labels from the target calibration split. The 900-row test split is disjoint from that calibration split in every row.

Protocol	Calibrator fit	Threshold fit	Zero-shot
P0 source-only	HaluEval validation (3,000)	HaluEval validation	yes
P1 target-calibrated	RAGTruth QA calibration (5,034)	fixed at 0.5	no
P2 target-thresholded	RAGTruth QA calibration (5,034)	same rows at 5% FPR	no

Both P1 and P2 metrics utilized the 900 rows of the test set. No calibration row used to fit either model overlaps this subset.

SHAP [19] TreeExplainers provide global explanations for the raw XGBoost model. There are three ways to determine whether these explanations can be trusted. First, the mean absolute SHAP ranking is compared with permutation importance via Kendall's tau correlation. Secondly, the top five feature set is re-estimated based upon 1,000 bootstrap resamples of the data, then the Jaccard index is computed between these two feature sets. Finally, two perturbation studies are performed on the test responses.

In content edits, entities, numbers, or dates are changed, supporting context is removed or an unrelated sentence is added to the content; each edit should affect the calibrated score if it alters the evidence. In structural edits, clause order is modified; the calibrated score should remain constant under such modifications. Four different metrics are computed for each type of operator.

Four metrics are computed for each operator: mean absolute score change, large-change rate, top-1 SHAP flip rate, and Spearman correlation.

A neutralization study determines how much the score assigned to each sample by a model will change once its k largest SHAP feature values have been reset to their respective medians. Mean changes are reported over samples for $k = 1, 3, 5, 10$.

Latency was determined on 200 test samples on a reference laptop equipped with an NVIDIA GeForce RTX 3060 6 GB GPU. Latency was also measured on the same 200 samples using GPT 5.6 Luna with the LLM-as-a-judge protocol of Zheng et al. [34], a commercial judge baseline, with a fixed prompt, and costs were calculated from the recorded token counts.

5. Results

5.1. Model comparison

The baselines are averaged over the three runs. For example, the F1 score of 0.9846 and AUROC of 0.9979 were both obtained when using the XGBoost method. These values are slightly greater than those achieved through the use of the random forest (by 0.0004) and greater than those obtained using logistic regression (by 0.0186). The heuristic (which finds its own threshold without any need for further training) yielded an F1 value of 0.9352, and the majority baseline (wherein the classifier always predicts the most common label) produced an F1 value of 0.0000. The NLI-only baseline scored very poorly at 0.6714. This shows that the other baselines captured some information about the lexical, length, and semantic properties of the input texts that was not available via the NLI cross-encoder alone. The answer-only TF-IDF model scored 0.9224, between the NLI-only and XGBoost results, while the full-input model scored 0.5999, below both.

Although the XGBoost margin over random forest is relatively small (only 0.0004 in F1), it was nominally significant in an uncorrected McNemar test comparing the paired seed-42 test predictions ($p = 0.044$). However, there were no such statistical differences when comparing XGBoost to logistic regression ($p = 0.43$) or the heuristic ($p = 0.35$).

A neural baseline trained solely on the source material outperforms EC-XGB in-domain (F1 0.9912 vs 0.9855), but fails completely to generalize across domains (RAGTruth AUROC = 0.337 vs EC-XGB AUROC = 0.582). Training an adapted encoder on exactly the same multi-source rows used to train EC-XGB reduces this domain-shift loss and raises the adapted encoder to 0.694, against EC-XGB's 0.582. This adaptation does not enable transfer learning, since neither the adapted encoder nor the original EC-XGB generalizes to the unseen QA task (adapted encoder = 0.490 vs original = 0.549), and even less so to FaithBench (adapted encoder = 0.434 vs original = 0.594). Adaptation improves any model's performance on seen tasks, whereas the feature model retains its relative ordering on unseen tasks.

Table 5
Test-set performance on HaluEval QA, mean over seeds 42/123/456 (n = 3,000). The ECE and Brier columns show the reliability of each model's risk score. The neural baseline is a fine-tuned RoBERTa-base with 256-token inputs. The majority baseline predicts the negative class for every row. Probability-based metrics cannot be calculated when the score is constant. The conventional row in this table and the m0 row in Table 6 come from separate tuning runs.

Model	P	R	F1	AUROC	PR-AUC	MCC	ECE	Brier
Heuristic (1−overlap)	0.9473	0.9233	0.9352	0.9132	0.8196	0.8723	0.3541	0.2672
Logistic regression	0.9768	0.9553	0.9660	0.9932	0.9901	0.9329	0.0099	0.0253
Random forest	0.9897	0.9789	0.9842	0.9980	0.9983	0.9687	0.0143	0.0127
NLI-only	0.6479	0.6967	0.6714	0.7177	0.7531	0.3189	0.0678	0.2182
TF-IDF (answer-only)	0.9550	0.8920	0.9224	0.9696	0.9665	0.8519	0.0831	0.0662
TF-IDF (full input)	0.6397	0.5647	0.5999	0.6754	0.6785	0.2484	0.0679	0.2328
XGBoost (conventional)	0.9932	0.9762	0.9846	0.9979	0.9983	0.9697	0.0045	0.0117
RoBERTa-base	0.9955	0.9869	0.9912	0.9990	0.9993	0.9825	0.0070	0.0081
Majority (negative)	0.0000	0.0000	0.0000	—	—	0.0000	—	—

5.2. Evidence-consistent XGBoost

Table 6 shows the results of the component ablation for EC-XGB on the HaluEval test set. The values are averages over the three seeds. The conventional XGBoost model (m0) reached an F1 of 0.9844, while the length-debias model with monotone constraints (m1) reached an F1 of 0.9852. The claim-level aggregates (m2) achieved an F1 of 0.9857 (+0.0005 above m1, within seed variability), and the full EC-XGB (m3) achieved an F1 of 0.9855 and an MCC of 0.9715.

There was no significant difference in the McNemar test (test of m0 vs. m3) comparing the paired predictions made by each model ($p = 0.45$). The Wilcoxon signed-rank test across the three seeds was underpowered ($p = 0.25$, the smallest attainable two-sided value with three pairs). Because the benchmark is saturated, removing some components serves mainly as a validation check that the components have been preserved.

When the monotone constraints were removed, there was no effect on the F1 score. However, the ECE increased by 0.0009, while the AUROC for RAGTruth-all decreased by 0.0043. When the source indicator was removed, the in-domain F1 score fell by less than 0.001 (the value went from 0.9855 to 0.9850). The removal changed the RAGTruth-all flag rate by only 0.2 percentage points, so the improvement due to multi-source training is not dependent upon the source indicator.

5.3. Calibration

The raw, Platt-scaled, and isotonic models are compared in Table 7: all three have a very low ECE (raw 0.0045, Platt 0.0091, isotonic 0.0070), but calibration does not improve on the raw model, and the HaluEval reliability plot confirms this.

Figure 4 shows the raw and target-calibrated score distributions behind the conventional-model rows. The practical constraint is the label budget: 252 labeled target rows already give the same test ECE as the full 5,034.

The source-only operating point (P0) is used to select the threshold on the HaluEval validation split that maximizes recall under a 5% false-positive budget. Recall stays between 99.0% and 99.5%, and the test false-positive rate stays near 5% as well. Table 8 puts both models on the same 900-row RAGTruth QA test set. Source calibration has little

Table 6
EC-XGB component ablation on the HaluEval test set, mean over seeds 42/123/456 (n = 3,000). m0 is the conventional XGBoost reference under the same protocol, m1 replaces the raw length counts with log-scaled values and adds monotone constraints (26 features), m2 adds the eight claim-level aggregates (34 features), and m3 adds the RAGTruth non-QA training rows and a binary source indicator (35 features) and is the deployed EC-XGB.

Variant	Features	F1	AUROC	MCC	ECE
m0 conventional	26	0.9844	0.9980	0.9693	0.0066
m1 debias	26	0.9852	0.9976	0.9708	0.0051
m2 + claims	34	0.9857	0.9984	0.9717	0.0078
m3 EC-XGB	35	0.9855	0.9984	0.9715	0.0074

Table 7
Calibration on the HaluEval test set (mean over seeds).

Method	F1	ECE	Brier	NLL
Raw XGBoost	0.9846	0.0045	0.0117	0.0449
Platt (sigmoid)	0.9848	0.0091	0.0129	0.0579
Isotonic	0.9844	0.0070	0.0124	0.0727

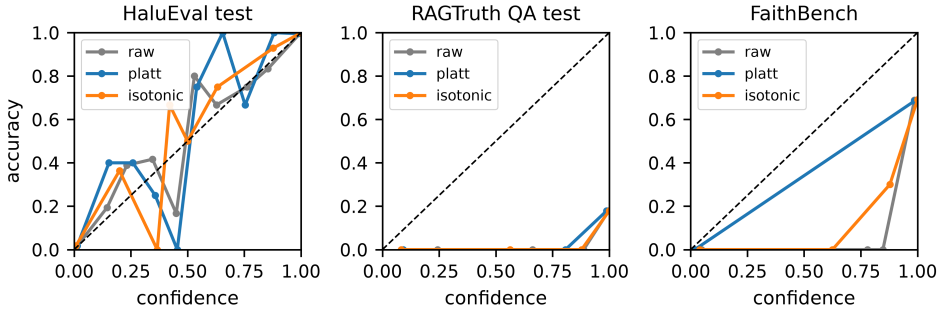


Figure 3: Reliability diagrams of the raw, Platt-calibrated, and isotonic models on the HaluEval test set (left), RAGTruth QA (middle), and FaithBench (right).

Table 8
Calibration on the same 900-row RAGTruth QA test set (17.8% positive). EC-XGB cells are means over seeds 42/123/456; conventional rows use the Table 5 checkpoint. The deployed EC-XGB display calibrator is the seed-42 target Platt row (ECE 0.126). AUROC is unchanged by calibration: 0.540 for the conventional model and 0.549 for EC-XGB.

Model	Calibrator	ECE	Brier	NLL	Pred. pos.
Conventional	Raw	0.8185	0.8164	5.20	0.999
Conventional	Source Platt (HaluEval)	0.8089	0.8011	3.62	0.999
Conventional	Target Platt (RAGTruth)	0.1335	0.1639	0.51	0.000
Conventional	Target isotonic	0.1316	0.1663	0.53	0.000
EC-XGB	Raw	0.7433	0.7061	2.91	0.974
EC-XGB	Source Platt (HaluEval)	0.7735	0.7527	3.18	0.975
EC-XGB	Target Platt (RAGTruth)	0.1263	0.1613	0.50	0.000
EC-XGB	Target isotonic	0.1260	0.1618	0.50	0.000

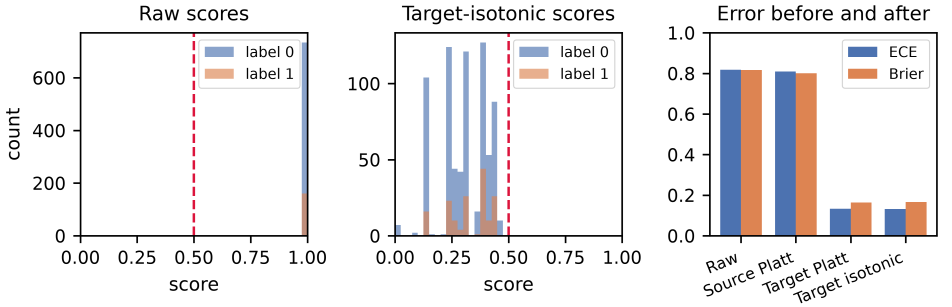


Figure 4: Target-domain recalibration of the conventional XGBoost model on RAGTruth QA. Left and middle: raw and target-calibrated score distributions on the 900-row test set, with the 0.5 threshold marked. Right: ECE and Brier before and after recalibration.

impact on the ECE, whereas target Platt calibration brings the ECE down to 0.1335 for the conventional model and 0.1263 for EC-XGB, with isotonic essentially tied. The target-calibrated display score (P1) was created using the target Platt fit for seed 42. This reduces the raw ECE from 0.726 to 0.126.

Every target-calibrated test score is below 0.5, so the fixed threshold predicts zero positives. The target-thresholded mode (P2) selects a threshold at a 5% false-positive budget (0.313) on the target calibration split. P2 achieves a test recall of 5.0% at a false-positive rate of 5.8% (precision 15.7%, F1 = 0.076; Wilson 95% intervals: recall [2.6, 9.6], FPR [4.3, 7.7]). The test rate exceeds the 5% target because the threshold is fixed on calibration. Target recalibration fixes the probability scale, not the ranking. Already 252 labeled target rows give a test ECE of 0.133.

5.4. Cross-domain transfer

The conventional model was trained on HaluEval only, while EC-XGB also saw RAGTruth summarization and data-to-text rows, so the EC-XGB results for those tasks measure adaptation (Table 9). Since 99.9% of the RAGTruth rows and 99.5% of the FaithBench rows exceeded the 0.5 threshold for the conventional model, there is little value in using the decision rule outside the benchmark. Compared with the conventional model, the EC-XGB method reduced the flag rate (the share of answers above 0.5) to 61.3% on the full RAGTruth test split and to 7.3% on FaithBench, and raised the AUROC from 0.475 to 0.582 on RAGTruth and from 0.509 to 0.594 on FaithBench. Furthermore, the ECE on RAGTruth decreased from 0.635 to 0.278. The held-out RAGTruth QA task is probably the most difficult test because none of its rows were included in the training set. There the F1 remained relatively constant (0.3022 vs. 0.3081). The ECE on the RAGTruth QA task decreased from 0.815 to 0.743, but the flat F1 shows that multi-source training does not improve accuracy on unseen tasks for free. For example, on FaithBench, the 0.5-threshold EC-XGB regime decreased the F1 from 0.813 to 0.120. However, the decrease in F1 is viewed as a limitation.

Table 9
Cross-domain behavior, mean over seeds. The conventional row is the XGBoost model trained on HaluEval only. Its external results are zero-shot. EC-XGB adds the RAGTruth non-QA training rows, so its summarization and data-to-text results measure adaptation, while its QA results are held out. RAGTruth all uses the 2,700-row test split, RAGTruth QA uses the 900-row held-out split, and FaithBench uses all 750 rows. Flagged is the share of rows above the 0.5 threshold.

Corpus	Model	F1	AUROC	Flagged	ECE
RAGTruth all	Conventional	0.5181	0.4753	99.9%	0.6346
RAGTruth all	EC-XGB	0.5608	0.5815	61.3%	0.2782
RAGTruth QA	Conventional	0.3022	0.5465	99.9%	0.8149
RAGTruth QA	EC-XGB	0.3081	0.5491	97.4%	0.7433
FaithBench	Conventional	0.8130	0.5089	99.5%	0.2917
FaithBench	EC-XGB	0.1201	0.5941	7.3%	0.4500

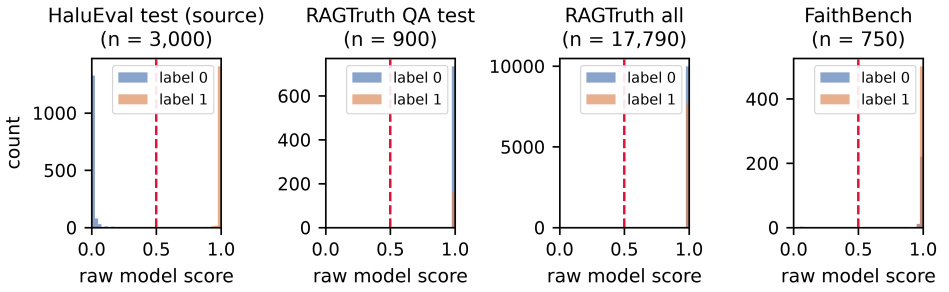


Figure 5: Raw model score distributions on the HaluEval test set and the external corpora, with the 0.5 threshold marked. The source panel separates the classes, while every external panel piles both labels at 1.0.

5.5. Ablation

The ablation study in Table 10 demonstrates how much each feature group contributes to the conventional XGBoost model by training a new version of the model with one group removed at a time. Most of the loss is due to lexical features (−0.0247 F1), while the contributions of the other feature groups (length and NLI) are somewhat smaller. There is little difference when excluding each of the entity, numeric, and hedging features, as all three remain within 0.0002 of the full model's F1 value. Removing the semantic feature group slightly improves F1.

Table 10
Feature-group ablation, mean over seeds (full model F1 = 0.9846, AUROC = 0.9979).

Removed group	F1	ΔF1	AUROC
length	0.9801	−0.0045	0.9963
lexical	0.9600	−0.0247	0.9923
entity	0.9847	+0.0001	0.9982
nli	0.9823	−0.0023	0.9973
numeric	0.9844	−0.0002	0.9982
hedging	0.9848	+0.0001	0.9981
semantic	0.9860	+0.0014	0.9982

5.6. Explanation reliability

Figure 6 presents the mean absolute SHAP values of the top-12 features. The overlap in contexts has the greatest effect, followed by the lengths of answers, then the contradictions between contexts and answers (in terms of probability), and lastly the cosine similarity between questions and answers. Three tests check whether the provided explanations are consistent. The SHAP rankings agree with the permutation importances at a Kendall tau of 0.66. The same five top features appear consistently after bootstrapping 1,000 times, yielding a Jaccard of 1.0. Reducing the predicted mean score monotonically from −0.079 at $k = 1$ to −0.412 at $k = 10$ when neutralizing the top-k features also indicates that these top features drive the predictions.

Table 11 provides the results of a perturbation test. Changes in content cause score changes according to how much each edit affects the semantics of the original input. On average, replacing an entity caused a 0.360 change in score; numbers caused a 0.345 change; dates caused a 0.279 change; and removing supporting context caused a 0.272 change. Irrelevant inserts as well as clause shuffles had almost no effect on the score, changing it by 0.006 and 0.001, respectively. Not once did the top-1 SHAP feature flip under any operator used in this test. Spearman correlations stayed in the range of 0.84 to 0.95 throughout all tests conducted. Date and clause-shuffle edits had only 13 and 14 samples, respectively.

A structured AI-assisted audit assessed the 26 exported explanations. The B5 export produced 26 error-enriched cases. These included 9 false positives, 10 false negatives, and 7 borderline cases in the 0.35 to 0.65 interval. Two review passes used an AI assistant. In both passes the AI assistant evaluated each explanation against the answer-context evidence, not the dataset label. The two passes agreed on 23 of the 26 explanations (88%). Each pass marked 15 explanations as implausible. False positives inherit the length confound. Many false negatives occur when a wrong

Table 11
Perturbation stability of the calibrated score and SHAP ranking.

Operator	n	Δscore	Large rate	Top-1 flip	Spearman
Entity replacement	78	0.360	0.372	0.000	0.840
Number replacement	22	0.345	0.364	0.000	0.852
Date replacement	13	0.279	0.308	0.000	0.870
Support removal	66	0.272	0.288	0.000	0.874
Irrelevant insert	120	0.006	0.008	0.000	0.948
Clause shuffle	14	0.001	0.000	0.000	0.921

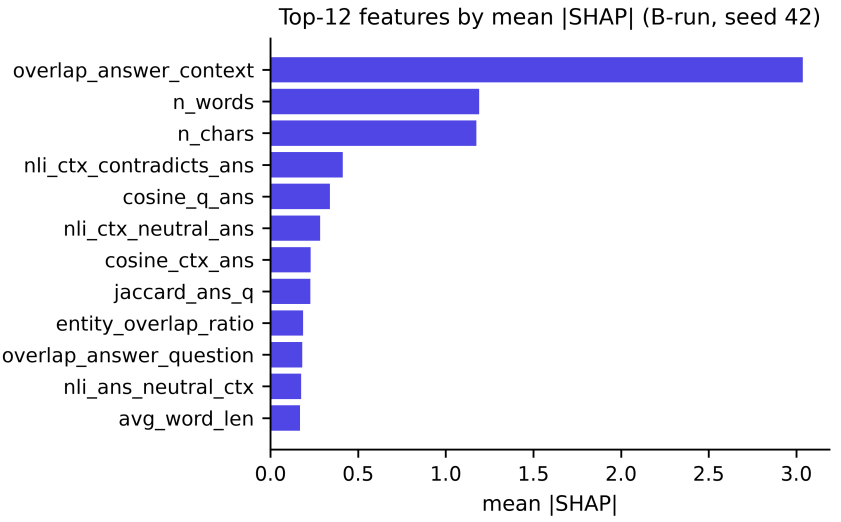


Figure 6: Mean absolute SHAP values for the top-12 features (B-run, seed 42).

answer reuses entities from its context, which keeps overlap high. There are two audited examples (item numbers 2251 and 2436) that have a hallucinated label for their correct answer although the context directly supports the answer. Therefore, these appear to be errors due to label noise in the benchmark rather than errors produced by the model. As the audit sample was intentionally populated with erroneous examples, the above percentages reflect the results of the audit process rather than those found in the test set.

5.7. Efficiency and the LLM judge

Table 12 displays the total average and distribution of the latency over the 200 test samples. The NLI cross-encoder and the spaCy entity pass were the slowest components in terms of latency with medians of 27.1 ms and 21.1 ms, respectively. The latency for XGBoost was much lower than that for either method, at 1.3 ms. SHAP had a somewhat greater latency to compute (1.9 ms), but the overall per-sample latency at the median was approximately 61.8 ms. The size of the deployed artifact was approximately 0.68 MB. The marginal cost is about $0.001 per 1,000 predictions. Local costs were based on the reference laptop with amortized hardware and electricity, so they are order-of-magnitude estimates in USD per 1,000 predictions. The GPT cost uses recorded token counts at $0.20 per million input tokens and $1.20 per million output tokens.

Table 13 puts every baseline on the same footing. The model-only p50 is timed per row over 200 test rows on the reference laptop, with classical models on CPU and NLI with embeddings on the GPU inside the shared feature pipeline. Classical heads cost almost nothing next to the 61.8 ms feature pipeline, and the two judges are three orders of magnitude slower than any local model. The overlap heuristic needs no model and costs 0.009 ms per row.

Table 14 compares EC-XGB with two separate judges on the same 200 balanced HaluEval rows, a subset of the 3,000-row headline test set, with 100 rows per class for 50% prevalence. GPT 5.6 Luna produced an F1 score of 0.846

Table 12
Per-component latency in ms (n = 200). Components are timed in separate instrumented passes over the same samples, so the component means need not sum to the end-to-end median. Retrieval and judge calls are excluded.

Component	p50	p95	mean
Core lexical	0.10	0.15	0.11
Entity (spaCy NER)	21.13	47.24	25.34
NLI (2 directions)	27.09	68.60	40.09
Semantic (MiniLM)	9.25	15.55	12.56
XGBoost inference	1.26	8.80	2.62
SHAP explanation	1.91	15.52	3.65
Total per sample	61.77		

Table 13
Per-baseline cost and latency. Size is the seed-42 artifact on disk. Feature models share the 61.8 ms pipeline end to end. The NLI-only head was not persisted, so its cost is the 27.1 ms NLI feature step plus a linear head of the same order as the timed logistic regression. * marks vectorizer-transform-only timings. RoBERTa inference was not timed because the weights were not kept, and its training cost is about 15 minutes per seed. Costs are USD per 1,000 predictions.

Baseline	Inputs	Size	Model p50 (ms)	Cost/1K
Heuristic (overlap)	text	—	0.009	≈$0
Logistic regression	26 feats	1.1 KB	0.001	≈$0.001
Random forest	26 feats	8.4 MB	0.331	≈$0.001
NLI-only	6 NLI probs	—	—	≈$0.001
TF-IDF (answer/full)	text	0.58/4.1 MB	0.014/0.110*	≈$0.001
XGBoost (conventional)	26 feats	0.49 MB	0.005	≈$0.001
RoBERTa-base	256 tokens	125M params	—	training only
EC-XGB (proposed)	35 feats	0.68 MB	0.008	≈$0.001
GPT 5.6 Luna judge	API	—	1443	$0.106
Qwen 3.5 4B judge	4-bit local	4B params	2623	$0

[0.781, 0.898], Qwen 3.5 4B produced 0.756 [0.667, 0.824], and EC-XGB produced 0.990. GPT agreed with EC-XGB on about 85.5% of the cases. The McNemar p-value for GPT vs. EC-XGB is 8.3×10^{-6}, with 27 cases where EC-XGB was correct and GPT was wrong, and 2 cases where GPT was correct and EC-XGB was wrong. An AI-assisted review of all 40 cases where Qwen was wrong found 20 answers that read as supported under a strict label, 6 ambiguous cases, 12 cases that Qwen missed but GPT caught, and 2 false alarms. A length-only baseline reached F1 0.905 on the same rows, so GPT does not offer an advantage over the length cue. GPT costs about $0.106 per 1,000 predictions and takes 1.44 s at the median. EC-XGB costs about $0.001 per 1,000 predictions and takes about 61.8 ms. The claims apply only to this dataset and these prompts.

Table 14
Two judges vs. EC-XGB on 200 balanced HaluEval rows (seed 42). GPT uses default decoding and Qwen is local 4-bit. Cost/1K is USD.

Model	Acc	P	R	F1	p50 (ms)	Cost/1K
GPT 5.6 Luna judge	0.865	0.987	0.740	0.846	1443	$0.106
Qwen 3.5 4B judge	0.800	0.969	0.620	0.756	2623	$0
EC-XGB (proposed)	0.990	0.990	0.990	0.990	62	≈$0.001

5.8. Error analysis

There were 48 errors in the 3,000-sample test set for the conventional calibrated XGBoost: 12 false positives and 36 false negatives, for an error rate of 1.6%. A heuristic analysis of 20 errors sampled evenly across the two directions (10 false positives and 10 false negatives) was grouped into two categories. These categories are (i) short-answer ambiguity, when answers are too short or ambiguous in terms of lexical or natural language inference (NLI) evidence, and (ii) label ambiguity, when answers are semantically possible under both labels. Seven false positives and four false

negatives were classified as label ambiguity. Similarly, three false positives and six false negatives were classified as short-answer ambiguity. The sample is balanced across the two error directions, so these counts describe the reviewed cases, not the full population.

The key aspect of the data is how the answer length of incorrect answers is distributed. Table 15 shows the false-positive and false-negative rates by answer length. The amount of hallucinations per answer length grows very quickly. For instance, 2.1% of one-word answers were hallucinations compared to 99.2% of answers longer than 17 words. Because of this relationship, answer length acts as a nearly perfect indicator of whether an answer will be labeled as a hallucination in HaluEval. The average score increased from 0.024 to 0.993 across the groupings. False positives occur mostly at the lower end: about 66.7% are in the five to eight word grouping. False negatives were most frequent by rate in the one to four word groups (about 18%), but the five to eight word group had the largest count (18 of 36). This mechanism explains the zero-shot transfer result. Natural language generation outputs are usually complete sentences, so the conventional model reads their length as evidence of hallucination. This is why recall was 100% while AUROC remained around 50%.

Several tests and controls support the claim that answer length drives performance. First, a logistic regression on the two answer length variables achieved F1 = 0.9457 and AUROC = 0.9759. Second, a reweighting control selected 247 examples from each of six answer length groups, for 1,482 examples. On this reweighted dataset, EC-XGB achieved F1 = 0.9867 and AUROC = 0.9987, while a length-only baseline achieved F1 = 0.9475 and AUROC = 0.9758. The difference is +0.039 F1 [0.029, 0.051] and +0.023 AUROC [0.016, 0.031], so EC-XGB still outperforms the length-only baseline after bin counts are equalized. This control does not prove that the shortcut is gone. Third, a length-matching control paired one positive example with one negative example at each answer length. This gave 163 pairs (326 examples) on the full test set, repeated 200 times at different sampling seeds. EC-XGB maintained AUROC = 0.9608 ± 0.0022, while the length-only baseline dropped to 0.5257 ± 0.0034, and the difference was 0.4351 [0.429, 0.442]. Finally, a length-conditional test removed the cue without subsampling. Within each exact-length stratum that contains both classes, length is constant, so the length-only model is at AUROC 0.5 by construction. Across the 12 such strata, EC-XGB reached a pair-weighted AUROC of 0.9773 ± 0.0009 over seeds.

Table 15
Test-set errors by answer length, mean over seeds (17+ contains only two negative rows).

Words	Rows	Hallucinated	FP rate	FN rate	Mean score
1	524	0.021	0.004	0.182	0.024
2	514	0.033	0.000	0.176	0.035
3–4	474	0.127	0.002	0.178	0.126
5–8	641	0.902	0.106	0.031	0.888
9–16	600	0.982	0.000	0.003	0.980
17+	247	0.992	0.333	0.000	0.993

5.9. Discussion

The informative results are outside the benchmark, where EC-XGB provides a better rank ordering of answers on RAGTruth than the conventional model: the conventional model identifies nearly all of them as unacceptable, and the display calibrator provides readable target scores. The gain coincides with multi-source training rather than a different type of learning system. A fine-tuned encoder still wins in-domain, so there is a trade-off between cost, transparency, and accuracy when choosing a feature-based model over other models.

6. Conclusion

A useful hallucination detector must be fast, inexpensive, and explainable. This research has created EC-XGB, an evidence-consistent XGBoost risk predictor. EC-XGB uses 26 evidence-consistency features, including log-scaled answer lengths under monotone constraints, 8 claim-level NLI aggregates, and a source indicator, with multi-source training, calibrated probabilities, and SHAP explanations. Using HaluEval QA data, this system achieves an F1 score of 0.9855 and an AUROC of 0.9984, comparable to conventional XGBoost performance on a saturated benchmark. The improvement appears on the RAGTruth task types used during training but not on unseen tasks. Conventional models flag 99.9% of all natural RAGTruth answers as being at-risk. EC-XGB flags 61.3% of all natural RAGTruth

answers as being at-risk. The AUROC increases from 0.475 to 0.582 while the ECE decreases from 0.635 to 0.278. Display calibrators reduce the RAGTruth QA ECE from 0.726 to 0.126 and require a labeled target sample (5,034 rows). Each analysis takes 61.8 ms and costs approximately $0.001 per 1,000 predictions, which is approximately 100 times less expensive than the GPT 5.6 Luna judge.

An encoder trained solely on source material outperforms all other models on the benchmark (F1 = 0.9912); however, the encoder performs poorly on unseen tasks (RAGTruth AUROC = 0.337 vs. 0.582). The raw model needs no calibration on HaluEval, and the entity and hedging features add no measurable value. The held-out RAGTruth QA task continues to have an F1 score close to 0.30. Therefore, this research views the supervised model as a domain-specific risk assessment tool and adaptation check, rather than a general-purpose detector, and the current deployment includes claim-level verification along with evidence retrieval. Future research will focus on multicalibration under stronger shifts, conformal prediction, transfer of thresholds across domains, and controlled evaluations on unseen tasks to detect shortcuts.

CRediT authorship contribution statement

Md. Atikur Rahaman: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing - original draft, Writing - review & editing. **Saiful Alam Sabbir:** Writing - review & editing. **MD. Miraz Ahamed:** Writing - review & editing. **Ohidujjaman Tuhin:** Supervision, Validation, Writing - review & editing.

Declarations

- **Funding:** No external funding was provided to conduct this research.
- **Conflicts of Interest:** The researchers declare no conflict of interest.
- **Data availability:** The raw evaluation datasets are not redistributed. The repository provides the download scripts, source revisions, SHA-256 hashes, and license notes for HaluEval (MIT), RAGTruth (MIT), and FaithBench (CC BY-NC-SA 4.0). Each split and feature matrix can be recreated from the original dataset sources. Model artifacts, split indices, and the frozen experiment manifest are included with the code release.
- **Code availability:** The training pipeline, FastAPI inference service, and Next.js dashboard are available in the project repository.
- **Ethical approval:** The study uses publicly available benchmark datasets under their original licenses. It involves no human participants and no personally identifiable information, so ethical approval is not required.

Acknowledgements

The authors thank the individuals responsible for releasing the datasets used in this study (HaluEval, RAGTruth, FaithBench) under open licenses.

References

[1] P. Manakul, A. Liusie, M. J. Gales, Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, in: Proceedings of EMNLP, 2023, pp. 9004–9017. doi:10.18653/v1/2023.emnlp-main.557.

[2] M. Belyi, R. Friel, S. Shao, A. Sanyal, Luna: A lightweight evaluation model to catch language model hallucinations with high accuracy and low cost, in: Proceedings of COLING (Industry Track), 2025, pp. 398–409. URL https://aclanthology.org/2025.coling-industry.34/

[3] S. Farquhar, J. Kossen, L. Kuhn, Y. Gal, Detecting hallucinations in large language models using semantic entropy, Nature 630 (8017) (2024) 625–630. doi:10.1038/s41586-024-07421-0.

[4] S. Yadav, N. K. Verma, A hybrid framework for hallucination detection in large language models, IEEE Transactions on Artificial Intelligence (2026). doi:10.1109/TAI.2026.3653354.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, ACM Computing Surveys 55 (12) (2023) 1–38. doi:10.1145/3571730.

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Transactions on Information Systems 43 (2) (2025) 1–55. doi:10.1145/3703155.

[7] A. Alansari, H. Luqman, Large language models hallucination: A comprehensive survey, Computer Science Review 61 (2026) 100970. doi:10.1016/j.cosrev.2026.100970.

[8] X. Cheng, J. Li, W. X. Zhao, H. Zhang, F. Zhang, F. Zhang, K. Gai, J.-R. Wen, Small agent can also rock! empowering small language models as hallucination detector, in: Proceedings of EMNLP, 2024, pp. 14600–14615. doi:10.18653/v1/2024.emnlp-main.809.

[9] S. Valentin, J. Fu, G. Detommaso, S. Xu, G. Zappella, B. Wang, Cost-effective hallucination detection for llms, arXiv preprint arXiv:2407.21424 (2024).
[10] D. Sundaragiri, L. M. Reddy, P. Gunavardhan, B. Navahith, Framework for hallucination detection in large language models, International Journal of Engineering Research & Technology (IJERT) 15 (04) (2026). doi:10.5281/zenodo.20025987.
[11] I. Haq, M. Saqib, Y. Zhang, I. A. Khan, Quantifying factual divergence in generative models: Shap-lime based hallucination score for llms, Multimedia Systems 32 (2) (2026). doi:10.1007/s00530-025-02150-4.
[12] Y. Deng, Z. Fang, S. Li, L. Chen, Beyond in-domain detection: Spikescore for cross-domain hallucination detection, in: Proceedings of ICLR, 2026.
[13] F. S. Bao, M. Li, R. Qu, et al., Faithbench: A diverse hallucination benchmark for summarization by modern llms, in: Proceedings of NAACL (Short Papers), 2025, pp. 448–461. doi:10.18653/v1/2025.naacl-short.38.
[14] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, J.-R. Wen, Halueval: A large-scale hallucination evaluation benchmark for large language models, in: Proceedings of EMNLP, 2023. doi:10.18653/v1/2023.emnlp-main.397.
[15] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, T. Zhang, Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models, in: Proceedings of ACL, 2024, pp. 10862–10878. doi:10.18653/v1/2024.acl-long.585.
[16] S. Lin, J. Hilton, O. Evans, Truthfulqa: Measuring how models mimic human falsehoods, in: Proceedings of ACL, 2022, pp. 3214–3252. doi:10.18653/v1/2022.acl-long.229.
[17] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, Factscore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of EMNLP, 2023, pp. 12076–12100. doi:10.18653/v1/2023.emnlp-main.741.
[18] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of ICML, 2017.
URL http://proceedings.mlr.press/v70/guo17a.html
[19] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.
URL https://papers.nips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html
[20] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD, 2016. doi:10.1145/2939672.2939785.
[21] Y. Bang, Z. Ji, A. Schelten, A. Hartshorn, T. Fowler, C. Zhang, N. Cancedda, P. Fung, HalluLens: LLM hallucination benchmark, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 24128–24156. doi:10.18653/v1/2025.acl-long.1176.
[22] P. He, J. Gao, W. Chen, DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.
[23] S. R. Bowman, G. Angeli, C. Potts, C. D. Manning, A large annotated corpus for learning natural language inference, in: Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2015, pp. 632–642. doi:10.18653/v1/D15-1075.
[24] A. Williams, N. Nangia, S. R. Bowman, A broad-coverage challenge corpus for sentence understanding through inference, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2018, pp. 1112–1122. doi:10.18653/v1/N18-1101.
[25] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992. doi:10.18653/v1/D19-1410.
[26] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, M. Zhou, MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers, in: Advances in Neural Information Processing Systems (NeurIPS), 2020.
[27] M. Honnibal, I. Montani, S. Van Landeghem, A. Boyd, spaCy: Industrial-strength natural language processing in Python (2020). doi:10.5281/zenodo.1212303.
[28] J. C. Platt, Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods, in: A. J. Smola, P. Bartlett, B. Schölkopf, D. Schuurmans (Eds.), Advances in Large Margin Classifiers, MIT Press, Cambridge, MA, 1999, pp. 61–74.
[29] B. Zadrozny, C. Elkan, Transforming classifier scores into accurate multiclass probability estimates, in: Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2002, pp. 694–699. doi:10.1145/775047.775151.
[30] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, B. Kim, Sanity checks for saliency maps, in: Advances in Neural Information Processing Systems (NeurIPS), 2018.
[31] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, J. Ye, INSIDE: LLMs' internal states retain the power of hallucination detection, in: International Conference on Learning Representations (ICLR), 2024.
[32] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, et al., RoBERTa: A robustly optimized BERT pretraining approach, arXiv preprint arXiv:1907.11692 (2019). doi:10.48550/arXiv.1907.11692.
[33] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, J. Snoek, Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift, in: Advances in Neural Information Processing Systems (NeurIPS), 2019.
[34] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023.